République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Djilali Liabès – Sidi Bel Abbès
Faculté des Sciences Exactes et Informatique
Département d'Informatique

TP N°1

Sécurité et Cryptographie Appliquée

Sujet :
Algorithme d'Euclide Étendu,
Inversion Modulaire
et Chiffrement Affine

Réalisé par :
Nom et Prénom : Benbarek Madjda

Groupe : 1
Spécialité : ISI

Année universitaire : 2025–2026

1 Objectifs pédagogiques

À l'issue de ce TP, on doit être capable de :

- Implémenter l'algorithme d'Euclide étendu sur de grands entiers.
- Calculer des inverses modulaires via une implémentation maîtrisée.
- Exploiter l'inversion modulaire dans la construction du chiffrement affine.
- Comprendre les contraintes mathématiques garantissant la réversibilité d'un chiffrement.

2 Rappels théoriques

2.1 Plus grand commun diviseur (PGCD)

Soient $a, b \in \mathbb{Z}$, non tous deux nuls. Le plus grand commun diviseur $\gcd(a, b)$ est le plus grand entier positif divisant simultanément a et b.

2.2 Identité de Bézout

Il existe toujours des entiers relatifs u et v tels que :

$$au + bv = \gcd(a, b)$$

L'algorithme d'Euclide étendu permet de calculer explicitement ces coefficients.

2.3 Inversion modulaire

Soient $a, n \in \mathbb{Z}$ avec $n \geq 2$. L'inverse modulaire de a modulo n existe si et seulement si $\gcd(a, n) = 1$. Dans ce cas :

$$au + nv = 1 \implies u \equiv a^{-1} \pmod{n}$$

2.4 Chiffrement affine

Pour un alphabet de taille n, le chiffrement et le déchiffrement sont :

$$E_{a,b}(m) = (am + b) \bmod n, \qquad D_{a,b}(c) = a^{-1}(c - b) \bmod n$$

avec la contrainte $\gcd(a, n) = 1$ pour garantir la réversibilité.

3 Exercice 1 : Classe `BigIntMath`

3.1 Description

La classe `BigIntMath` regroupe deux méthodes statiques :

- `extendedGCD(a, b)` : retourne le triplet (d, u, v) tel que $d = \gcd(a, b)$ et $au + bv = d$.
- `modInverse(a, n)` : retourne $u = a^{-1} \bmod n$, ou lève une exception si l'inverse n'existe pas.

Python utilise des entiers à précision arbitraire nativement, ce qui satisfait la contrainte d'utilisation de grands entiers.

3.2 Code source

```python
class BigIntMath:
    """
    Classe regroupant les operations arithmetiques fondamentales :
    - Algorithme d'Euclide etendu
    - Inversion modulaire
    Utilise les entiers natifs Python (precision arbitraire).
    """

    @staticmethod
    def extendedGCD(a: int, b: int) -> tuple:
        """
        Algorithme d'Euclide etendu.
        Retourne (d, u, v) tel que d = gcd(a,b) et a*u + b*v = d.
        """
        r, r_prime = a, b
        u, u_prime = 1, 0
        v, v_prime = 0, 1

        while r_prime != 0:
            q = r // r_prime
            r, u, v, r_prime, u_prime, v_prime = (
                r_prime,
                u_prime,
                v_prime,
                r - q * r_prime,
                u - q * u_prime,
                v - q * v_prime,
            )


        if r < 0:
            r, u, v = -r, -u, -v

        return (r, u, v)

    @staticmethod
    def modInverse(a: int, n: int) -> int:
        """
        Calcule l'inverse modulaire de a modulo n.
        Retourne u dans {0, ..., n-1} tel que a*u = 1 (mod n).
        Leve ValueError si gcd(a, n) != 1.
        """
        if n < 2:
            raise ValueError(f"Le modulus n doit etre >= 2, recu : {n}")

        d, u, _ = BigIntMath.extendedGCD(a % n, n)

        if d != 1:
```

```python
            raise ValueError(
                f"Pas d'inverse : gcd({a}, {n}) = {d} != 1"
            )

        return u % n
```

Listing 1 – BigIntMath.py – Euclide étendu et inversion modulaire

3.3 Explication des commandes clés

Instruction	Rôle
`r, r_prime = a, b`	Initialisation des restes $r_0 = a,\ r_1 = b$
`u, u_prime = 1, 0`	Coefficients de Bézout initiaux pour a
`v, v_prime = 0, 1`	Coefficients de Bézout initiaux pour b
`q = r // r_prime`	Division entière $q = \lfloor r/r' \rfloor$
`while r_prime != 0`	Boucle jusqu'au reste nul (fin de l'algorithme)
`r, u, v, ...`	Mise à jour simultanée des 6 variables en une ligne
`if r < 0: r,u,v = -r,-u,-v`	Normalisation du signe du PGCD
`if d != 1: raise ValueError`	Vérification de la condition d'inversibilité
`return u % n`	Ramène le résultat dans $\{0, \ldots, n-1\}$

3.4 Résultats obtenus

Listing 2 – Test extendedGCD et modInverse

```
TEST  Algorithme d'Euclide Etendu
a = 35, b = 15
gcd(35, 15) = 5
u = 1, v = -2
Verification : 35*1 + 15*(-2) = 5  OK

a = 240, b = 46
gcd(240, 46) = 2
u = -9, v = 47
Verification : 240*(-9) + 46*47 = 2  OK

a = 17, b = 13
gcd(17, 13) = 1
u = -3, v = 4
Verification : 17*(-3) + 13*4 = 1  OK

 TEST  Inversion Modulaire
3^(-1) mod 26 = 9
Verification : 3 x 9 mod 26 = 1  OK

7^(-1) mod 26 = 15
```

```
Verification : 7 x 15 mod 26 = 1  OK

5^(-1) mod 26 = 21
Verification : 5 x 21 mod 26 = 1  OK

modInverse(4, 26) -> Pas d'inverse : gcd(4, 26) = 2 != 1
```

4 Exercice 2 : Classe `AffineCipher`

4.1 Description

La classe `AffineCipher` implémente le chiffrement affine en utilisant exclusivement l'inversion modulaire développée dans `BigIntMath`. Elle prend en paramètres la clé multiplicative a, la clé additive b et la taille de l'alphabet n (26 par défaut pour l'alphabet latin).

4.2 Code source

```python
class AffineCipher:
    """
    Chiffrement affine sur un alphabet de taille n.

    Chiffrement :  E(m) = (a*m + b) mod n
    Dechiffrement : D(c) = a_inv * (c - b) mod n

    a_inv est calcule via l'algorithme d'Euclide etendu.
    """

    def __init__(self, a: int, b: int, n: int = 26):
        """
        Initialise le chiffrement affine.
        Leve ValueError si gcd(a, n) != 1 (cle invalide).
        """
        self.n = n
        self.b = b % n


        d, _, _ = BigIntMath.extendedGCD(a % n, n)
        if d != 1:
            raise ValueError(
                f"Cle invalide : gcd({a}, {n}) = {d} != 1. "
                f"Choisissez a tel que gcd(a, {n}) = 1."
            )
        self.a = a % n


        self.a_inv = BigIntMath.modInverse(self.a, self.n)

    def encrypt(self, m: int) -> int:
```

```python
        """ Chiffre un entier m dans {0,...,n-1}. """
        return (self.a * m + self.b) % self.n

    def decrypt(self, c: int) -> int:
        """ Dechiffre un entier c dans {0,...,n-1}. """
        return (self.a_inv * (c - self.b)) % self.n

    def encrypt_text(self, plaintext: str) -> str:
        """
        Chiffre un texte (lettres uniquement).
        Les caracteres non alphabetiques sont conserves tels quels.
        """
        result = []
        for ch in plaintext.upper():
            if ch.isalpha():
                m = ord(ch) - ord('A')
                c = self.encrypt(m)
                result.append(chr(c + ord('A')))
            else:
                result.append(ch)
        return ''.join(result)

    def decrypt_text(self, ciphertext: str) -> str:
        """
        Dechiffre un texte (lettres uniquement).
        Les caracteres non alphabetiques sont conserves tels quels.
        """
        result = []
        for ch in ciphertext.upper():
            if ch.isalpha():
                c = ord(ch) - ord('A')
                m = self.decrypt(c)
                result.append(chr(m + ord('A')))
            else:
                result.append(ch)
        return ''.join(result)
```

Listing 3 – AffineCipher.py – Chiffrement et déchiffrement affine

4.3 Explication des commandes clés

Instruction	Rôle
`self.a_inv = modInverse(a, n)`	Calcule $a^{-1} \bmod n$ via Euclide étendu
`(a * m + b) % n`	Formule de chiffrement $E(m) = am + b \bmod n$
`(a_inv * (c - b)) % n`	Formule de déchiffrement $D(c) = a^{-1}(c - b) \bmod n$
`ord(ch) - ord('A')`	Convertit une lettre en entier $\in \{0, \ldots, 25\}$
`chr(m + ord('A'))`	Reconvertit un entier en lettre
`ch.isalpha()`	Ignore les caractères non alphabétiques
`if d != 1: raise ValueError`	Rejette une clé a non inversible

4.4 Vérification : $D(E(m)) = m$

La propriété fondamentale du chiffrement est sa réversibilité. On vérifie :

$$D(E(m)) = a^{-1}\big((am + b) - b\big) \bmod n = a^{-1} \cdot am \bmod n = m$$

4.5 Résultats obtenus

Listing 4 – Test du chiffrement affine

```
TEST  Chiffrement Affine
[Entiers] a=5, b=8, n=26
D(E(m)) = m pour tout m dans {0..25}  OK

[Texte] a=7, b=3, n=26
Original  : HELLO
Chiffre   : AFCCX
Dechiffre : HELLO  OK

Original  : CRYPTOGRAPHIE
Chiffre   : RSPEGXTSDEAHF
Dechiffre : CRYPTOGRAPHIE  OK

Original  : UNIVERSITE
Chiffre   : NQHUFSZHGF
Dechiffre : UNIVERSITE  OK

Original  : BONJOUR MONDE
Chiffre   : KXQOXNS JXQYF
Dechiffre : BONJOUR MONDE  OK

[Cle invalide] a=4, b=3, n=26
Exception attendue :
Cle invalide : gcd(4, 26) = 2 != 1.
Choisissez a tel que gcd(a, 26) = 1.  OK
```

5 Code complet

```python

class BigIntMath:
    @staticmethod
    def extendedGCD(a: int, b: int) -> tuple:
        r, r_prime = a, b
        u, u_prime = 1, 0
        v, v_prime = 0, 1
        while r_prime != 0:
            q = r // r_prime
            r, u, v, r_prime, u_prime, v_prime = (
                r_prime, u_prime, v_prime,
                r - q*r_prime, u - q*u_prime, v - q*v_prime,
            )
        if r < 0:
            r, u, v = -r, -u, -v
        return (r, u, v)

    @staticmethod
    def modInverse(a: int, n: int) -> int:
        if n < 2:
            raise ValueError(f"n doit etre >= 2, recu : {n}")
        d, u, _ = BigIntMath.extendedGCD(a % n, n)
        if d != 1:
            raise ValueError(f"Pas d'inverse : gcd({a},{n})={d}!=1"
                )
        return u % n


class AffineCipher:
    def __init__(self, a: int, b: int, n: int = 26):
        self.n = n
        self.b = b % n
        d, _, _ = BigIntMath.extendedGCD(a % n, n)
        if d != 1:
            raise ValueError(
                f"Cle invalide : gcd({a},{n})={d}!=1. "
            )
        self.a = a % n
        self.a_inv = BigIntMath.modInverse(self.a, self.n)

    def encrypt(self, m: int) -> int:
        return (self.a * m + self.b) % self.n

    def decrypt(self, c: int) -> int:
        return (self.a_inv * (c - self.b)) % self.n

    def encrypt_text(self, plaintext: str) -> str:
        res = []
        for ch in plaintext.upper():
```

```python
            if ch.isalpha():
                res.append(chr(self.encrypt(ord(ch)-ord('A'))+ord('A')))
            else:
                res.append(ch)
        return ''.join(res)

    def decrypt_text(self, ciphertext: str) -> str:
        res = []
        for ch in ciphertext.upper():
            if ch.isalpha():
                res.append(chr(self.decrypt(ord(ch)-ord('A'))+ord('A')))
            else:
                res.append(ch)
        return ''.join(res)



def test_extended_gcd():
    print("\n== TEST -- Algorithme d'Euclide Etendu ==")
    exemples = [(35,15),(240,46),(17,13),(100,75)]
    for a, b in exemples:
        d, u, v = BigIntMath.extendedGCD(a, b)
        check = a*u + b*v
        ok = "OK" if check == d else "ERREUR"
        print(f"gcd({a},{b})={d}, u={u}, v={v} [{ok}]")

def test_mod_inverse():
    print("\n== TEST -- Inversion Modulaire ==")
    exemples = [(3,26),(7,26),(5,26),(4,26)]
    for a, n in exemples:
        try:
            inv = BigIntMath.modInverse(a, n)
            check = (a * inv) % n
            ok = "OK" if check == 1 else "ERREUR"
            print(f"{a}^(-1) mod {n} = {inv} [{ok}]")
        except ValueError as e:
            print(f"modInverse({a},{n}) -> {e}")

def test_affine():
    print("\n== TEST -- Chiffrement Affine ==")
    cipher = AffineCipher(a=7, b=3, n=26)
    msgs = ["HELLO","CRYPTOGRAPHIE","UNIVERSITE","BONJOUR MONDE"]
    for msg in msgs:
        enc = cipher.encrypt_text(msg)
        dec = cipher.decrypt_text(enc)
        lettres_in  = ''.join(c for c in msg.upper() if c.isalpha())
        lettres_out = ''.join(c for c in dec if c.isalpha())
        ok = "OK" if lettres_in == lettres_out else "ERREUR"
```

```python
        print(f"[{ok}] {msg} -> {enc} -> {dec}")
    try:
        AffineCipher(a=4, b=3)
    except ValueError as e:
        print(f"[Cle invalide] {e}")


if __name__ == "__main__":
    test_extended_gcd()
    test_mod_inverse()
    test_affine()
    print("\nTous les tests termines avec succes.")
```

Listing 5 – tp1_cryptographie.py – Code complet

6 Conclusion

Dans ce TP, nous avons :

1. Implémenté l'algorithme d'Euclide étendu qui calcule, en plus du PGCD, les coefficients de Bézout u et v vérifiant $au + bv = \gcd(a, b)$.
2. Construit l'inversion modulaire à partir de cet algorithme : l'inverse $a^{-1} \bmod n$ est directement le coefficient $u \bmod n$, sous réserve que $\gcd(a, n) = 1$.
3. Appliqué ces résultats au chiffrement affine et vérifié expérimentalement que $D(E(m)) = m$ pour toutes les lettres de l'alphabet.

Lien entre les trois notions : le chiffrement affine repose entièrement sur l'inversibilité de la clé a ; l'inversion modulaire fournit cet inverse ; et l'algorithme d'Euclide étendu est le moteur qui calcule cet inverse efficacement, même pour des entiers très grands.

1/2